例題 01-1　BPE のマージを手計算する

これは何？ 何に使う？

何か　テキストをトークンに分ける方法の 1 つ。よく一緒に出てくる文字のペアを繰り返しくっつけて、語彙（トークンの一覧）を作る。

何のため　語彙サイズを抑えながら、知らない単語も知っている部品に分解して扱うため。GPT や Llama などほとんどの LLM が BPE 系のトークナイザを使っている。

問題

次のコーパス（単語: 出現回数）で BPE（Byte Pair Encoding）を学習する。

hug	pug	pun	bun	hugs
10	5	12	4	5

初期語彙は文字 {b, g, h, n, p, s, u} とする。

(1) 最初の 3 回のマージで作られるトークンを順に答えよ。

(2) 学習後、未知語 **"bugs"** と **"mug"** はどうトークン化されるか。

解説

Step 1: 隣り合う文字ペアの出現回数を数える

各単語を文字に分け、ペアごとに「その単語の出現回数」を足し合わせる。

```
u g : hug 10 + pug 5 + hugs 5 = 20  ← 最多
p u : pun 12 + pug 5 = 17    u n : pun 12 + bun 4 = 16
h u : hug 10 + hugs 5 = 15  g s : 5    b u : 4
```

→ 1 回目のマージは **u + g → "ug"**

Step 2: "ug" を 1 つの記号として数え直す

```
h ug, p ug, p u n, b u n, h ug s
u n : 12 + 4 = 16  h ug : 15  p u : 12  p ug : 5  ug s : 5
```

→ 2 回目のマージは **u + n → "un"**

Step 3: もう一度数え直す

```
h ug, p ug, p un, b un, h ug s
h ug : 15  p un : 12  p ug : 5  ug s : 5  b un : 4
```

→ 3 回目のマージは **h + ug → "hug"**

Step 4: 新しい単語は「学習した順」にマージを適用する

```
bugs : b u g s → (u+g) b ug s → これ以上適用できない
mug  : m u g   → (u+g) m ug   → "m" は語彙にない → [UNK]
```

解答

(1) ① **ug** (20回)　② **un** (16回)　③ **hug** (15回)

(2) "bugs" → **[b, ug, s]**　/　"mug" → **[[UNK], ug]**

★ BPE は「最も頻出するペアを 1 つにまとめる」を繰り返すだけ。
実用の Byte-level BPE は 256 種のバイトを初期語彙にするので [UNK] が出ない。